"将注意力旋转90°"：深入浅出解读 Kimi 最新出圈成果

原创 鹅厂程序员 腾讯技术工程 2026年3月24日 02:49 广东 132人



作者：dorianbian

前几天，一篇来自Kimi的论文「ATTENTION RESIDUALS」在 AI 圈引发了激烈讨论——马斯克罕见地发出评价："Impressive work from Kimi"。



同时，两位前Openai大佬也同样发出了高度评价，OpenAI 「推理模型之父」Jerry Tworek表示“深度学习2.0时代即将到来”。



Openai 创始团队成员之一Andrej Karpathy看过此文后感叹，“我们对Attention is All You Need的理解还是片面了”。



然而这篇论文的共同一作者之一，是一位17岁的中国高中生——陈广宇。那这篇论文究竟写了什么，能让这些顶级大佬集体出动，一切要从 Transformer 的一个"祖传问题"说起。

周末仔细读了这份成果，这里用更易懂的方式让大家了解它是什么，解决了什么问题。（论文链接在文章底部）

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号

背景

目前主流的LLM模型主要采用标准残差作为transformer层间的连接，这种连接方式让深度模型的落地具备可行性。但是标准残差中每一层都接受此前所有层输出，并将其等权求和的性质也带来了以下两个问题：

1. 随着深度的加深，每一层的贡献被逐层稀释
2. 隐状态值爆炸

特别是第一个问题，将直接影响模型理解与处理复杂任务的能力上限。为了解决以上问题，那么需要把每一层的固定权重，修改为每一层应具有加权权重，提升更高价值层的贡献，以提升模型表达能力的上限。本文提出的“注意力残差（ATTENTION RESIDUALS）”机制，即通过注意力机制，构建transformer每层和前序层的关联权重，也就是把“注意力旋转了90°，应用在网络的深度轴上”。

注意力残差

在介绍本文提出的Full Attention Residuals和折中方案Block Attention Residuals之前，先举一个形象的例子来辅助理解两种机制和Standard Residual的区别：

设想数学的学习路径，小学数学（共6册）→中学数学（共3册）→高中数学（共3册）→高等数学（共2册），全部学习完成后，用全部数学知识解决现实中遇到的问题。这里我们把每一册类比

为模型的“层”，把每个阶段的合集类比为Block Attention Residuals机制中的“块”，那么三种机制的类比应该如下：

Standard Residual：把全14册数学书中的每一册整理出等量的知识点，放在一起用于解决具体问题。——这样问题的解决率一定是偏低的，原因是更多现实中的问题应更依赖高等数学中的知识点，但高等数学中的知识点的整体占比是偏低的，这就类比了信息逐层稀释。

Full Attention Residuals：把全14册数学书中的每一册整理出的知识点按照权重分配，高等数学最多，高中数学次之，小学和中学数学相对少。——也就是实际解决问题时更多使用高等数学的知识点，那么问题的解决率会更高，类比了每一层贡献加权求和。

Block Attention Residuals：但是全14册逐册整理并加权合并的工作量大，那么把内容分块整理，也就是分别把小学数学、中学数学、高中数学和高等数学各自整理成知识点摘要，同时每一部分摘要整理过程中，要参考前序整理完的摘要，最终使用的时候采用更多的高等数学摘要中的知识点来解决问题。——类比了Block AttnRes和Full AttnRes开销和效果的权衡。

三种残差机制原理示意图如下，供理解下文中三种机制理论原理时参考使用：

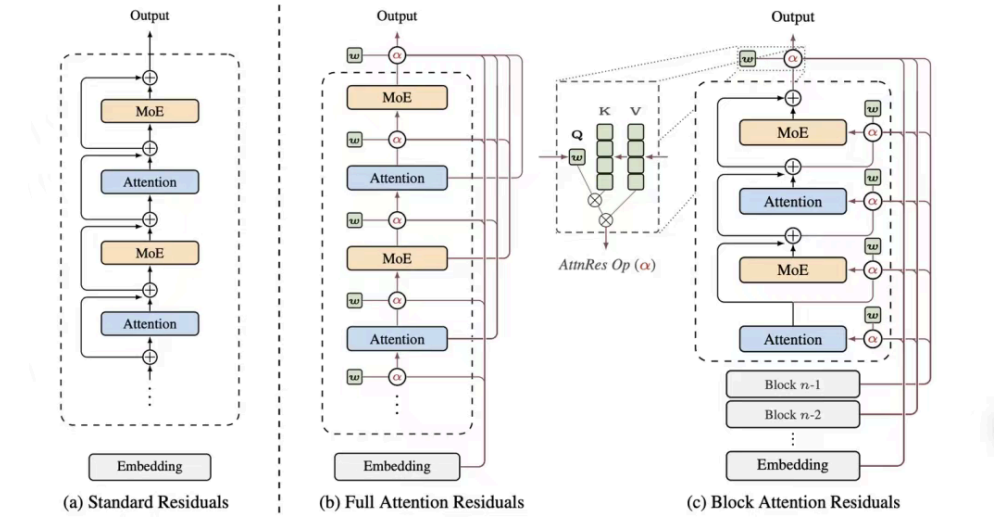


Figure 1: Overview of Attention Residuals. (a) Standard Residuals: standard residual connections with uniform additive accumulation. (b) Full AttnRes: each layer selectively aggregates all previous layer outputs via learned attention weights. (c) Block AttnRes: layers are grouped into blocks, reducing memory from $O(Ld)$ to $O(Nd)$.

标准残差（Standard Residual）

定义如下图公式：

$$h_l = h_{l-1} + f_{l-1}(h_{l-1})$$

也就是每一层的输入隐状态=前一层的输入隐状态+前一层变换（attn或ffn层）。我们将这个公式逐层展开，同时令h1=v0（v代表每一层的变换结果f(h)，v0就是原始embedding后的结果用于模型输入），得到如下公式：

$$\boldsymbol{h}_l = \sum_{i=0}^{l-1} \boldsymbol{v}_i$$

也就是说，每一层的输入，等于此前每一层变换结果的累积和，这也是标准残差的一条优雅性质。同时上述公式也说明了，历史每一层相较于深层的贡献是均等的，这就是为什么标准残差会

带来贡献逐层稀释的原因。

全注意力残差（Full Attention Residuals）

为了解决标准残差每层贡献权重均等带来的问题，本文提出了「全注意力残差」的机制，即把每层的贡献添加权重，让更重要的层具备更强的表示能力。这里依然v代表每一层的变换结果f(h)，v0就是原始embedding后的结果用于模型输入，即h1。每一层输入隐状态公式如下：

$$\boldsymbol{h}_l = \sum_{i=0}^{l-1} \alpha_{i \to l} \cdot \boldsymbol{v}_i$$

同时，每一层的权重参数是由注意力机制计算得到，最终得到历史所有层对该层的贡献权重softmax值，公式如下：

$$\alpha_{i \to l} = \frac{\phi(\boldsymbol{q}_l, \boldsymbol{k}_i)}{\sum_{j=0}^{l-1} \phi(\boldsymbol{q}_l, \boldsymbol{k}_j)}$$

此处q是可学习的查询向量，k=v做了简化处理，意图是直接查询历史层输出的值的相似度，也避免了额外维护一组k向量。

$$\boldsymbol{q}_l = \boldsymbol{w}_l, \quad \boldsymbol{k}_i = \boldsymbol{v}_i = \begin{cases} \boldsymbol{h}_1 & i = 0 \\ f_i(\boldsymbol{h}_i) & 1 \le i \le l-1 \end{cases}$$

由此则实现了“每一层输入由更高关联度的历史层输出加权得到”的目的，从根本上解决了重要层贡献被稀释的问题。

分块注意力残差（Block Attention Residuals）

全注意力残差机制虽然从理论上解决了逐层贡献稀释的问题，但在工程实践中，以下两个问题依然非常棘手：

内存开销：在主流的大规模训练中，往往采用“激活值重计算”的方式来节省显存。针对标准残差，只需要保留最近两层的激活值即可，但全注意力残差需要必须保留每一层的过程激活值。

通信开销：在主流的流水线并行策略下，模型将会被切分为多个阶段，这意味着所有的层的过程激活值都要被跨阶段传输，通信开销不可控。

基于以上问题，提出了折中方案「分块注意力残差」的机制，通过把L层分成N个块，每个块聚合为单一表征，供后续阶段注意力查询使用，达到保证效果和开销之间权衡的目的。我们分两个部分来理解这个机制：

块内聚合：

先定义块内“累加和”，用于表示单块内聚合的结果，公式如下：

$$\boldsymbol{b}_n = \sum_{j \in \mathcal{B}_n} f_j(\boldsymbol{h}_j)$$

其中bn表示第n个块中，该块内在计算第j+1层前时的累加和，当块内所有层执行完成，参考上述公式，此时累加和=每一层变化结果的无加权和，也就得到了该块的聚合结果。

块间注意力：

块间注意力的具体表现是：块内每一层计算输入的时候，需要查询「原始输入」、「已完成的历史块的聚合结果」和截止当前的「累加和」，对这几部分内容作注意力加权计算。用于当前层输入隐状态计算的V矩阵的定义如下，q、k及注意力权重公式完成参照全注意力残差模式中的定义：

$$\mathbf{V} = \begin{cases} [\boldsymbol{b}_0, \boldsymbol{b}_1, \ldots, \boldsymbol{b}_{n-1}]^\top & \text{if } i = 1 \text{ (first layer of block } n) \\ [\boldsymbol{b}_0, \boldsymbol{b}_1, \ldots, \boldsymbol{b}_{n-1}, \boldsymbol{b}_n^{i-1}]^\top & \text{if } i \geq 2 \text{ (subsequent layers)} \end{cases}$$

其中定义b0=h1（也就是原始embedding后的输入），每个块内第一层只查询「历史输入」cat「每一个历史块的聚合结果」，第二次开始再额外cat该块当前的「累加和」。

最终，该块内每一层的输入hl的公式推导依然符合如下形式，其中α代表第m模块输出到第l层的贡献权重：

$$\boldsymbol{h}_l = \sum_m \alpha_{m \to l} \cdot \boldsymbol{v}_m$$

带入V，也就是

When $i = 1$ (first layer in block n):

$$\boldsymbol{h}_l = \alpha_{0 \to l} \boldsymbol{b}_0 + \alpha_{1 \to l} \boldsymbol{b}_1 + \cdots + \alpha_{n-1 \to l} \boldsymbol{b}_{n-1}$$

When $i \geq 2$ (later layers in block n):

$$\boldsymbol{h}_l = \alpha_{0 \to l} \boldsymbol{b}_0 + \cdots + \alpha_{n-1 \to l} \boldsymbol{b}_{n-1} + \alpha_{n \to l} \boldsymbol{b}_n^{i-1}$$

当该块内所有层都计算完成，自然也得到了该块内聚合的结果bn，用于后续块的每一层计算时使用。

如此设计则大幅改善内存和通信开销的问题，同时文中提出，超参数N（块数）设为8时，可以达到对比全注意力残差几乎同样的效果，同时将开销可控。

讨论与进一步分析

几种残差机制结构化矩阵处理

我们把几种主流的残差机制统一抽象成如下形式，通过对权重矩阵M来分析不同机制。其中，v0=h1，vi=fi(hi)，具体如下：

$$\boldsymbol{h}_l = \sum_{i=0}^{l-1} \mathbf{M}_{i \to l} \, \boldsymbol{v}_i$$

其中以下所有机制中的M矩阵均以4*4为例。

1、Standard Residual

根据前文中标准残差的公式：

$$\boldsymbol{h}_l = \sum_{i=0}^{l-1} \boldsymbol{v}_i$$

我们得到M权重矩阵，如下：

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

可以清晰看到，标准残差前序每一层的贡献权重均为固定的值，也就是1。

2、Highway

一种在标准残差基础上略微优化的残差机制，第l层输入的定义如下：

$$\boldsymbol{h}_l = (1-g_l)\,\boldsymbol{h}_{l-1} + g_l\, f_{l-1}(\boldsymbol{h}_{l-1})$$

这种机制的特点是把前一层的输入和当前层的变换的两个权重用门控系数控制，我们根据上述公式推导得到M权重矩阵如下：

Highway

$$\begin{bmatrix} 1 & & & \\ \gamma^{\times}_{1\to 2} & g_2 & & \\ \gamma^{\times}_{1\to 3} & g_2\,\gamma^{\times}_{2\to 3} & g_3 & \\ \gamma^{\times}_{1\to 4} & g_2\,\gamma^{\times}_{2\to 4} & g_3\,\gamma^{\times}_{3\to 4} & g_4 \end{bmatrix}$$

其中定义了一种连乘的表示方法：

$$\gamma^{\times}_{i\to l} := \prod_{j=i+1}^{l}(1-g_j)$$

例如，2→4代表i=2，l=4，连乘计算=（1-g3）（1-g4），对应上图中矩阵的各个元素的表示方法。

这种机制改善了标准残差中各层权重的灵活性，但在实践中表现效果依然有限。

3、(m)HC

这种机制的核心思路是将标准残差的“单流传递”改为多流传递，作用也是提升模型的表达能力，同时也降低梯度爆炸/消失的概率，第l层的隐状态定义如下：

$$\mathbf{H}_l = \mathbf{H}_{l-1}\mathbf{A}_l + f_{l-1}(\mathbf{H}_{l-1}\boldsymbol{\alpha}_{l-1})\,\boldsymbol{\beta}_{l-1}^{\top},$$

注意这里定义的不是层输入隐状态hl，而是多流隐状态Hl，他们真正的关系是（即输入隐状态需要将多流隐状态先降为单流）：

$$\mathbf{h}_l := \mathbf{H}_l \boldsymbol{\alpha}_l$$

根据定义的公式推导得到M权重矩阵如下：

$$\begin{bmatrix} \boldsymbol{\beta}_0^\top \boldsymbol{\alpha}_1 & & & \\ \boldsymbol{\beta}_0^\top \mathbf{A}_{1\to 2}^{\times} \boldsymbol{\alpha}_2 & \boldsymbol{\beta}_1^\top \boldsymbol{\alpha}_2 & & \\ \boldsymbol{\beta}_0^\top \mathbf{A}_{1\to 3}^{\times} \boldsymbol{\alpha}_3 & \boldsymbol{\beta}_1^\top \mathbf{A}_{2\to 3}^{\times} \boldsymbol{\alpha}_3 & \boldsymbol{\beta}_2^\top \boldsymbol{\alpha}_3 & \\ \boldsymbol{\beta}_0^\top \mathbf{A}_{1\to 4}^{\times} \boldsymbol{\alpha}_4 & \boldsymbol{\beta}_1^\top \mathbf{A}_{2\to 4}^{\times} \boldsymbol{\alpha}_4 & \boldsymbol{\beta}_2^\top \mathbf{A}_{3\to 4}^{\times} \boldsymbol{\alpha}_4 & \boldsymbol{\beta}_3^\top \boldsymbol{\alpha}_4 \end{bmatrix}$$

上矩阵中每个元素的推导结果为：

$$\mathbf{M}_{i\to l} = \boldsymbol{\beta}_i^\top \, \mathbf{A}_{i+1\to l}^{\times} \, \boldsymbol{\alpha}_l,$$

该机制显著改善层间贡献权重的灵活性，但也需要维护更多的超参数和更大的计算开销。

4、Full Attention Residuals

前章节已详细介绍，其M权重矩阵如下：

$$\begin{bmatrix} \phi(\boldsymbol{w}_1, \boldsymbol{k}_0) & & & \\ \phi(\boldsymbol{w}_2, \boldsymbol{k}_0) & \phi(\boldsymbol{w}_2, \boldsymbol{k}_1) & & \\ \phi(\boldsymbol{w}_3, \boldsymbol{k}_0) & \phi(\boldsymbol{w}_3, \boldsymbol{k}_1) & \phi(\boldsymbol{w}_3, \boldsymbol{k}_2) & \\ \phi(\boldsymbol{w}_4, \boldsymbol{k}_0) & \phi(\boldsymbol{w}_4, \boldsymbol{k}_1) & \phi(\boldsymbol{w}_4, \boldsymbol{k}_2) & \phi(\boldsymbol{w}_4, \boldsymbol{k}_3) \end{bmatrix}$$

可以清晰看到，每一层对后续层的权重系数全是由注意力计算得到的，是完全相互独立的，灵活性最高，虽额外计算开销有限，但额外的内存和通信开销很棘手。

5、Block Attention Residuals

前章节已详细介绍，我们以每块2层为例，其M权重矩阵如下：

$$\begin{bmatrix} \phi(\boldsymbol{w}_1, \boldsymbol{k}_0) & & & \\ \phi(\boldsymbol{w}_2, \boldsymbol{k}_0) & \phi(\boldsymbol{w}_2, \boldsymbol{k}_1) & & \\ \phi(\boldsymbol{w}_3, \boldsymbol{k}_0) & \phi(\boldsymbol{w}_3, \boldsymbol{k}_1 + \boldsymbol{k}_2) & & \\ \phi(\boldsymbol{w}_4, \boldsymbol{k}_0) & \phi(\boldsymbol{w}_4, \boldsymbol{k}_1 + \boldsymbol{k}_2) & \phi(\boldsymbol{w}_4, \boldsymbol{k}_3) & \end{bmatrix}$$

其中：

第一行，只查询了原始输入的注意力；

第二行，查询了原始输入和块内当前累加和的注意力；

第三行，查询了原始输入+第一块聚合的注意力；

第四行，查询了原始输入+第一块聚合+块内当前累加和的注意力；

图中虚线框表示非块内首行，也就是需要cat块内当前累加和查询注意力

如有第五行和第六行，则应为：

第五行，查询了原始输入+第一、第二块聚合的注意力；

第六行，查询了原始输入+第一、第二块聚合+块内当前累加和的注意力。另外这里应有虚线框（非块内首行）；

这种机制是文中最终权衡了开销和效果的最佳注意力残差机制，确保层权重参数丰富性的同时，尽可能减小开销。

抽象分析

通过抽象权重矩阵来统一分析上述几种形式的残差设计，可以通过权重矩阵的性质分析残差机制的灵活性，而灵活性越高意味着模型的表达能力限制更少。这里通过上述五种机制中权重矩阵M的半可分秩进行分析：

残差机制	Standard residual	Highway	(m)HC	Full AttnRes	Block AttnRes
半可分秩	1	1	最大m	L	[N,N+S]

其中：

L代模型的层数

m为(m)HC机制中的m个并行流，同时定义中的转移矩阵A也是m*m的矩阵

N代表Block AttnRes机制中N分块，每个分块S层

权重参数矩阵M的半可分秩本质上是**模型在深度轴上信息聚合模式的 “自由度”**，半可分秩越高，层之间权重分配的灵活性越高。例如，标准残差是表固定的权重为1，它的M矩阵半可分秩为1；而Full AttnRes每一层都有独立的权重参数，它的M矩阵半可分秩为满秩。最终本文中，为了权衡效率、开销和效果，把Block AttnRes作为最佳选择，它的半可分秩介于「块数」~「块数+层数」之间。

写在最后

目前模型能扮演的角色已经不仅仅是“chat bot‘了，而是可以处理现实中复杂任务的现代智能体的中枢系统。当智能体在处理复杂任务的过程中，往往需要经过多轮复杂交互，包括对话、终端执行操作、工具调用、GUI交互等类型，同时伴以越来越长、越来越复杂的上下文，这就要求模型具备能真正理解和有效处理更长、更复杂上下文的能力，本文中提出的「注意力残差机制」，就是提升模型对复杂上下文理解能力上限的可行路径之一。

既然「Attention is All You Need」，那为什么不把注意力再旋转90°，让层间连接也使用注意力机制呢？

论文链接：https://arxiv.org/pdf/2603.15031，非常推荐大家仔细研读。





阅读 9927

留言 8

写留言

Reconnecting 广东 3月24日 11

90°是个很神奇的数字，数学里面i的意义就是旋转90°

边冀 Dorian 浙江 3月24日 3

不羁的学习 广东 3月24日

@元宝 用通俗易懂的语言解释一下

元宝 广东 3月24日 7

传统AI模型处理信息像背书，每层都花同样力气。Kimi的新"注意力残差"技术让模型学会"划重点"，自动加强关键层的贡献。就像学霸复习时不平均用力，而是重点攻克难点，这让AI理解复杂问题的能力大幅提升。

内容由AI生成

Haunru 北京 3月25日

这就像将一本巨著分成 小学篇、中学篇、大学篇 等章节。你不需要记住书中每一个字，只需要记住每一章的核心摘要。在解决新问题时，你可以快速翻阅这些摘要，并关注大学篇的摘要。论文发现，当分成8块时，其效果几乎可以媲美开销巨大的全注意力残差，同时开销可控，可以真正用于训练超大模型。



Haunru 北京 3月25日 1

注意力旋转90度 的比喻和权重矩阵的半可分秩分析，将复杂的工程改进用一个清晰、统一的数学框架呈现出来，极具美感。



边冀 Dorian 浙江 3月25日

这也是我在读这篇论文时候最被吸引的点之一



BJCcc 河北 3月24日

有关注意力残差的深度好文👍